

[4]

15 (a) Compare ISO 9126 / CMM / Boehm models

→

	ISO 9126	Capability maturity Model	Boehm's Quality model
Primary focus is	Product Quality. It defines a framework for measuring the software product itself.	Process Maturity. It evaluates the evolution and capability of the organization's software development processes.	Hierarchical Trade-offs. It emphasizes a hierarchical structure of attributes to evaluate complex systems and trade-offs.
Structure	6 - Characteristics :- Functionality, reliability, Usability, Efficiency, Maintainability, Portability.	5 - Maturity levels Initial, repeatable, defined, managed, optimizing.	7 - Characteristics Utility, reliability, efficiency, Human engineering, testability, Maintainability, Portability.
Key - Methodology.	Provides a standardized checklist to ensure the software meets specified requirements and performs under conditions.	Uses Key Process Areas and moves organizations from chaotic (level 1) to optimized (level 5) workflows.	Categorizes software into three levels (As-is - Utility, Maintainability, General utility) to assess utility and maintainability simultaneously.
Main Goal	Standardization of software quality evaluation.	Process enhancement, resource optimization, & Benchmarking.	Allows stakeholders to evaluate trade-offs between different quality attributes.
Example	Checking if the login page loads in under 2 sec's to satisfy Efficiency see q. note	checking if the development team has documented plan for tracking bugs to ensure it is repeatable.	Decide to see/learn software today (As-is) to gain immediate sales even if the code is messy and hard to update later (Maintainability).

(b) Recommendation for a Fast-growing Start-up

↳ Use the ~~Bohm~~ Boehm's Quality Model (with Agile).

→ Why CMM is NOT a good fit.

↳ Too heavy for startups — CMM requires lots of documentation, defined processes, and expertise.

Startups usually do not have resources for this.

↳ Conflicts with agility — CMM level 3+ demands formal, organization-wide processes, while start-ups rely on speed, flexibility, and minimal documentation.

↳ Start-ups naturally operate at CMM level 1: Processes are informal and chaotic; forcing maturity too early slows MVP delivery.

→ Why Boehm's Model is good fit.

↳ Perfect for trade-offs: Start-ups constantly balance new features, bugs, and technical debt. Boehm's model helps make these trade-offs explicit.

↳ Matches early product stages: Ideal where rapid iteration matters more than quality.

↳ Supports speed: The idea of AS-IS utility aligns with releasing usable versions quickly, even if maintainability is lower.

→ Where ISO 9126 fits.

↳ Useful as a checklist for non-functional requirements (e.g. performance).

↳ But pursuing full compliance too early slows down fast experimentation.

→ Summary -

1) CMM - Like requiring full factory docs. before building racecar → Too slow

2) ISO 9126 - A spec sheet ensuring the car has essential parts - helpful but not strategic.

34 Borkum's Model - A strategy talk deciding what to optimize
(speed vs durability) to win the race.
exactly what a startup needs.

classmate

Date _____

Page _____

16

Compact project plan - Hospital Management System. (HMS)① Project Overview & Process Model

Goal - Develop a centralized HMS to manage patient records, appointments, billing, clinical workflows, and inventory with high reliability and security.

Process model - Agile (Scrum)

↳ ~~Health~~ Healthcare environments experience frequent requirement changes, involve many user roles, and require iterative validation with clinicians.

Scrum roles

- ↳ Product Owner - Hospital representative
- ↳ Scrum master
- ↳ Development team

Workflow

- ↳ Sprint length : 2-4 weeks
- ↳ Deliverable : Potential shippable increment
- ↳ Artifacts : Product backlog, sprint backlog, burn down chart.

(i) Requirement Analysis

A] Functional requirements

① Patient management

- Register, update, search patients.
- Store demographics, history, allergies.

② Appointment Scheduling

- Doctor availability, slot booking, reminders

③ Clinical / EMR Module

- Record diagnosis, prescriptions, lab results

④ Billing & Insurance

- Generate invoices, apply insurance rules, track payments

⑤ Pharmacy & Inventory

- Track medicines, stock updates, expiry alerts

⑥ Role-Based Access Control

- Admin, doctor, nurse, receptionist, accountant.

B] Non-Functional Requirements

① Security - Data encryption, audit logs, secure authentication

② Reliability - 24/7 uptime, fault-tolerant design.

③ Performance - Key pages load ≤ 2 seconds.

④ Interoperability - Integrates with lab devices, external HIS/EMR APIs

⑤ Scalability - Support multi-branch hospitals.

(ii) Architecture & Design

A) System Modules

- Admin Module - User roles, permissions, settings.
- Clinical Module - EMR, prescriptions, lab management
- Appointment Module - Scheduling, calendars, notifications
- Financial Module - Billing, insurance, accounts.
- Pharmacy Module - Inventory, dispensing.
- Reporting Module - Dashboards, analytics.

B) Logical Data Model (Sample classes).

- Patient (Patient-ID, Name, DOB, History)
- Doctor (Doctor-ID, Specialization, Schedule)
- Appointment (Appt-ID, Date, Patient-ID, Doctor-ID, Status)
- Bill (Bill-ID, Items, Amount, Insurance Details).

Relationships:-

- A doctor has multiple appointments.
- A patient has many visits/Records.

C) UI/UX Design

Key screens:-

- Login
- Dashboard
- Patient Registration
- Appointment scheduler
- Billing Form.

(iii) Cost & effort estimationA] COCOMO (Intermediate) - Semi-detached projectAssumptions :-

- Estimate size: 50 KLOC
- Required reliability: High → increase EAF
- Team experience: Moderate

Effort formula :-

$$E = a \times (\text{KLOC})^b \times \text{EAF}$$

For semi-detached :-

- $a = 3.0$
- $b = 1.12$

Base Effort :-

$$3.0 \times 50^{1.12} \approx 240 \text{ PM}$$

Assuming $\text{EAF} \approx 1.32$ (High reliability, complexity):

$$240 \times 1.32 \approx 316 \text{ PM}$$

Schedule Duration :

$$D = 2.5 \times E^{0.35} \approx 18 \text{ months.}$$

B] PUTNAM (SLRM) Model - optional

- ↳ Useful when manpower is constrained. Effort follows a Rayleigh curve.
- Shortening schedule dramatically increases peak staffing and cost.

(iv) Risk & Mitigation

Risk	Impact	Mitigation
Requirement changes	High	Agile sprints, continuous feedback
Integration with legacy systems	Med/High	Early prototyping, dedicated integration team.
Security breaches	Critical	Pen-testing, encryption, RBAC.
Data mitigation issues	Med	Automated mitigation scripts, validation tools
Downtime in clinical settings	High	Failover servers, load balancing.

(v) Release Strategy :-

1) <u>Version 1.0 - MVP</u> → Patient registration → Appointments → Billing basics → Minimal infrastructure & documentation. → Goal: Fast deployment & validation.	3) <u>Version 3.X - Optimization</u> → Scalability improvements. → Performance tuning. → Reporting & analytics.
2) <u>Version 2.X - Expansion</u> → EMR integration → Lab & pharmacy modules → Improved UI/UX. → Structured testing, branching strategy.	4) <u>LTS (Long-term Support)</u> . → Security fixes → Stability improvements → No major feature additions.

(vi) Maintenance Plan

- Corrective : Bug fixes, incident response.
- Adaptive : Update due to OS, device, or regulatory changes.
- Perfective : UI improvements, faster queries, new reports.
- Preventive : Code refactoring, documentation, logs cleanup.
- Legacy Support : Data archiving, mitigation to next-gen HMS.